

GUÍA RESUMEN — Resolución de Problemas Algorítmicos

Resumen integral para resolver problemas de algoritmos tipo HackerRank/LeetCode sin volver a leer el módulo completo. Hilo conductor: Framework de 6 pasos → Big-O → Reconocimiento de patrones → Templates en Dart → Recursión → System Design.

0 El Protocolo: Framework de 6 pasos

SIEMPRE EN ORDEN

1 Entender "¿Qué me dan? ¿Qué me piden?"	2 Constraints "¿Qué complejidad necesito?"	3 Ejemplos normal + edge + tricky	4 Patrón "¿Qué algoritmo encaja?"	5 Pseudocódigo explicarlo en palabras	6 Implementar testear 3 casos + optimizar
---	---	--	--	--	--

Regla de oro: "Si no puedes explicar el problema con tus propias palabras, no lo entiendes." Leer el enunciado **2 veces** y reformular antes de tocar el teclado.

Paso	Qué haces	Clave	Dónde aprender más
1. Entender	Entrada, salida, términos, restricciones implícitas	Reformular en voz alta	01-metodologia-general.md
2. Constraints	Leer límites antes de diseñar	Regla del 10 ⁸ ops/seg	02-analisis-complejidad.md
3. Ejemplos	1 normal + 1 edge case + 1 tricky, a mano	vacío · 1 elem · negativos · duplicados	01-metodologia-general.md
4. Patrón	Nombrar el patrón antes de codificar	80% cae en ~15 patrones	03-reconocimiento-patrones.md
5. Pseudocódigo	1 paso por línea, nombres descriptivos	Validar lógica sin sintaxis	06-comunicacion-y-pseudocodigo.md
6. Implementar	Early returns, estructura correcta, testear	happy path + edge + stress	01-metodologia-general.md

NO hagas: ✗ **codificar sin entender** · ✗ **ignorar constraints** · ✗ **probar solo el happy path** · ✗ **adivinar el patrón sin señales** · ✗ **saltar del enunciado al código.**

Big-O mide **cómo crece el tiempo** cuando aumentan los datos. No es velocidad absoluta: clasifica el tipo de crecimiento.

Las 4 complejidades esenciales

Complejidad	Nombre	Analogía	En código
O(1)	Constante	Abrir en una página marcada	<code>arr[0] · map[key]</code>
O(n)	Lineal	Leer un libro página por página	<code>1 for que recorre todo</code>
O(n²)	Cuadrática	Comparar cada libro con todos	<code>2 for anidados</code>
O(log n)	Logarítmica	Diccionario: mitad por mitad	<code>while (n > 1) { n = n ~/ 2; }</code>

Pregunta clave: si duplicas n y el tiempo se duplica → O(n); se cuadruplica → O(n²); apenas cambia → O(log n); no cambia → O(1).

Regla del 10⁸ y tabla de constraints → complejidad

Un procesador ejecuta ~100 millones de operaciones/segundo. Si tu algoritmo supera eso → "Time Limit Exceeded".

Si el problema dice...	Tu algoritmo debe ser...
$n \leq 10$	Cualquier cosa (backtracking / O(2 ⁿ))
$n \leq 100$	O(n³) o mejor
$n \leq 5,000$	O(n²) o mejor (2 loops anidados)
$n \leq 10^5$	O(n log n) o mejor (sorting / binary search)
$n \leq 10^6$	O(n) o mejor (un solo recorrido)
$n \leq 10^7$	O(n) con operaciones simples

Cómo analizar cualquier código: 3 patrones

Patrón en el código	Complejidad	Señal
1 loop	O(n)	<code>for (cada elemento) { hacer algo }</code>
2 loops anidados (mismos datos)	O(n²)	<code>for i { for j { ... } }</code>
while que divide por 2	O(log n)	<code>while (n > 1) { n = n ~/ 2; }</code>

Reglas de combinación: secuencial (uno tras otro) → toma la MÁS GRANDE · **anidados** → se MULTIPLICAN · **constante dentro del loop** → no cambia. O(n) + O(n²) = O(n²).

Operaciones O(1) vs O(n)

O(1)	O(n) — ¡cuidado!
Suma, resta, módulo, comparación	<code>arr.contains(x) · arr.indexOf(x)</code>
<code>arr[i] · map[key] · set.contains(x)</code>	<code>arr.insert(0, x) · arr.removeAt(i)</code>
Asignar, imprimir	<code>List.from(arr) · arr.sublist()</code>

Espacio y recursión

Complejidad **espacial**: array/map de n → O(n); recursión profunda → O(n) stack (puede causar stack overflow); variable auxiliar → O(1).

Si cada llamada hace...	Complejidad	Ejemplo
1 llamada recursiva con n-1	O(n)	Factorial
1 llamada recursiva con n/2	O(log n)	Binary search
2 llamadas recursivas	O(2 ⁿ) — lento	Fibonacci sin memo
2 llamadas + ordenar	O(n log n)	Mergesort

Regla: recursión que parece loop (1 llamada) = O(n)/O(log n). Que parece árbol (2+ llamadas) = O(2ⁿ) o peor → necesita memoización.

Checklist pre-código: ¿cuál es el n máximo? · ¿qué complejidad necesito? · ¿mi enfoque cumple? · ¿puedo usar Map/Set para O(1)? · ¿necesito pre-computar? · ¿cuánto espacio extra uso?

No inventes algoritmos: **reconoce** cuál patrón aplica. Framework de 3 pasos:

- 1. **Identificar tipo de input** → patrones candidatos
- 2. **Detectar keywords** del enunciado → patrón
- 3. **Validar contra las constraints** → ¿cabe la complejidad?

Paso 1: Tipo de Input → Patrones

Input	Patrones a considerar primero
Array ordenado	Two Pointers, Binary Search
Array desordenado	HashMap, Sorting + Two Pointers, Sliding Window
String	HashMap (frecuencias), Sliding Window, Trie
Linked List	Two Pointers (fast/slow), Reversal
Tree	DFS, BFS, Recursión
Graph	BFS, DFS, Topological Sort, Union-Find
Matrix / Grid	BFS + visited, DFS, DP
Intervalos	Sorting + Greedy, Merge logic
Stream / Online	Heap / Priority Queue
Números / Bits	Bit manipulation, Math, Binary Search

Paso 2: Keywords → Patrón

Keyword del enunciado	Patrón
subarray/substring contiguo · longest/shortest	Sliding Window
par / triplet que suma	Two Pointers o HashMap
sorted array · buscar en rango	Binary Search
minimum steps / shortest path (sin peso)	BFS
all combinations / permutations	Backtracking
k-th largest / smallest · top K	Heap / Priority Queue
overlapping subproblems + optimal	Dynamic Programming
ciclo / cycle	DFS + visited o Union-Find
connected / componentes	DFS / BFS / Union-Find
next greater / smaller element	Monotonic Stack
dependency / prerequisite	Topological Sort
frequency / count duplicates	HashMap
merge intervals / overlapping	Sorting + Intervals
prefix matching	Trie

Paso 3: Validación

¿La complejidad del patrón cabe en las constraints? ($O(n^2)$ cuando necesitas $O(n)$ → mal patrón). ¿Cubre todos los edge cases? (con negativos, Sliding Window falla → usa Prefix Sum). ¿Hay uno más simple? (Greedy > DP si ambos funcionan).

El Mantra: "Este es un problema de **[PATRÓN]** porque el input es **[TIPO]** y me piden **[QUÉ]**, con complejidad **[O(?)]**." Si puedes completarlo, estás listo para implementar.

Necesito...	Usa...	Complejidad
Acceso rápido por índice	List	O(1) acceso · O(n) búsqueda
Verificar si un valor existe	Set	O(1) contains
Frecuencias / pares / agrupar	Map<K,V>	O(1) lookup
Procesar FIFO / BFS	Queue	O(1) add/remove
Min/max repetidamente · Top-K	PriorityQueue (Heap)	O(log n) add/remove
Buscar en datos ordenados	Binary Search sobre List	O(log n)
Conexiones dinámicas	Union-Find (propia)	O(α(n)) ≈ O(1)
Prefix matching (strings)	Trie (propia)	O(L)

Complejidad de operaciones en Dart

Operación	Complejidad	Detalle
arr.sort() (Tim Sort)	O(n log n)	Default de Dart, estable
set.union · intersection	O(n) · O(min(n,m))	—
map.putIfAbsent · update	O(1)	Patrón de frecuencias: (freq[item] ?? 0) + 1
heap.removeFirst()	O(log n)	Extrae el mínimo (min-heap por defecto)

Patrón Top-K con Heap: min-heap de tamaño K → agregas y si supera K, removeFirst(). Al final el heap contiene los K más grandes en O(n log k) en vez de O(n log n).

Decisiones de estructura por problema

Problema	Estructura óptima	Por qué
Two Sum	Map<int,int>	Lookup de complemento O(1)
Sliding Window Máximo	Deque	Monotonic, O(1) por operación
K-th Largest	PriorityQueue (min-heap k)	O(n log k)
Detectar ciclo en lista	Two Pointers (fast/slow)	O(1) espacio, sin visited
Subarray Sum = k	Map + prefix sum	O(n) un solo pass
Range sum queries	List precomputada	O(1) por query tras O(n)

#	Patrón	Cuándo usar	Complejidad	Error común
1	Sliding Window	Subarray/substring contiguo (longest/shortest, k)	O(n)	Usarlo con negativos (rompe monotonicidad → Prefix Sum)
2	Two Pointers	Array ordenado, pares/triplets, partition	O(n)	No ordenar el array primero
3	BFS	Camino mínimo sin pesos, niveles, grid	O(V+E)	No marcar visitados antes de agregar a la cola
4	DFS	Todos los caminos, componentes, ciclos, islas	O(V+E)	No hacer backtrack / deshacer cambios
5	Binary Search	Array ordenado, espacio de respuesta monotónico	O(log n)	Off-by-one: confundir left < right vs left <= right
6	Greedy	Óptimo local = global (sin subproblemas superpuestos)	O(n log n)	Asumir que siempre funciona — hay que probarlo
7	DP	Subproblemas superpuestos + optimalidad	O(n) a O(n·m)	No definir bien la recurrence relation
8	Backtracking	Todas las combinaciones/permutaciones (n ≤ 20)	O(2^n)/O(n!)	Olvidar removeLast() / used[i] = false

Plantillas esenciales (condensadas)

```
// TWO POINTERS (array ordenado) - O(n) | O(1)
while (left < right) {
    int sum = arr[left] + arr[right];
    if (sum == target) return [left, right];
    else if (sum < target) left++; // necesito más
    else right--; // necesito menos
}

// SLIDING WINDOW (tamaño variable, longest) - O(n)
int left = 0, maxLen = 0;
for (int right = 0; right < arr.length; right++) {
    // expandir: agregar arr[right] al state
    while (/* condición violada */) { left++; /* quitar arr[left] */ }
    maxLen = max(maxLen, right - left + 1);
}
```

```
// BFS (camino mínimo sin peso) - O(V+E) | O(V)
Queue<List<int>> queue = Queue(); // [node, dist]
Set<int> visited = {};
queue.add([start, 0]); visited.add(start);
while (queue.isNotEmpty()) {
    var cur = queue.removeFirst();
    if (cur[0] == target) return cur[1];
    for (int nb in graph[cur[0]]) {
        if (!visited.contains(nb)) {
            visited.add(nb);
            queue.add([nb, cur[1] + 1]);
        }
    }
}
return -1; // sin camino

// DFS grid (contar islas) - O(rows*cols)
void dfs(Grid g, int r, int c) {
    if (fuera de rango || g[r][c] != '1') return;
    g[r][c] = '0'; // marcar visitado
    dfs(g, r+1, c); dfs(g, r-1, c); dfs(g, r, c+1); dfs(g, r, c-1);
}
```

```
// BINARY SEARCH - O(log n) | O(1)
while (left <= right) {
    int mid = left + (right - left) ~/ 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
}
return -1;

// DP (Climbing Stairs) - O(n) | O(n)
List<int> dp = List.filled(n + 1, 0);
dp[1] = 1; dp[2] = 2;
for (int i = 3; i <= n; i++) { dp[i] = dp[i-1] + dp[i-2]; }
return dp[n];

// BACKTRACKING (permutaciones) - O(n!)
if (current.length == nums.length) { resultado.add(List.from(current)); return; }
for (int i = 0; i < nums.length; i++) {
    if (used[i]) continue;
    used[i] = true; current.add(nums[i]); // hacer
    backtrack(); // explorar
    current.removeLast(); used[i] = false; // deshacer + CLAVE
}
```

Estructura de presentación (5 min)

1 Reformular 30 seg	2 Constraints 30 seg	3 Brute force 30 seg	4 Óptimo 1 min	5 Pseudocódigo 2 min
---------------------------	----------------------------	----------------------------	----------------------	----------------------------

1. "Entiendo que me pides [reformulación]." · 2. "El input llega a n=[valor], así que necesito [complejidad]." · 3. "Lo obvio sería [brute force] O(?), pero es lento porque [razón]." · 4. "Puedo mejorarlo con [patrón]: la idea es [2 oraciones]." · 5. Pseudocódigo limpio.

Reglas de pseudocódigo limpio

- 1 paso por línea · nombres descriptivos (`queue` , `visited` , no `q` , `v`)
- Indentación para bloques (while, for, if)
- Comentarios solo para la lógica clave · separa datos de lógica

Mantra de entrevista: "Elegí [patrón] porque [señal del enunciado], complejidad [O(?) tiempo, O(?) espacio], edge cases [casos], alternativas [por qué no otras]."

Errores de comunicación

Error	Cómo evitarlo
Codificar en silencio	Narra cada paso mientras escribes
No preguntar antes	"¿Puedo asumir que [asunción]?"
No mencionar brute force	Siempre menciónalo primero
Olvidar edge cases	Menciona al menos 2 antes de codificar
No decir la complejidad	Termina con "La complejidad es O(?)"

Recursión: los 3 pasos

1. **Caso base** — ¿cuándo para? Sin esto, llamada infinita.
2. **Caso recursivo** — ¿cómo reduces el problema?
3. **¿Qué cambia en cada llamada?** — algo debe acercarse al caso base.

```
int resolver(tipo input) {  
  if (input es casoBase) return resultadoBase; // 1. base  
  return operacion(input, resolver(inputReducido)); // 2. recursivo  
}
```

Memoización: evitar recalcular

Recursión con **2+ llamadas** recalcula lo mismo (fibonacci = O(2^n)). Con `Map` o `@cache` → O(n). Regla: si ves 2+ llamadas recursivas, necesitas memo.

Cuándo usar cada uno

Situación	Usar
Dividir y conquistar (merge sort)	Recursión simple
Explorar todas las combinaciones	Backtracking
Subproblemas repetidos	Recursión + memoización
Árboles binarios	Recursión (DFS)

Backtracking = probar, y si no funciona, REVERTIR. Plantilla: ¿es solución? → agregar y return. Para cada candidato: si es válido → hacer · explorar · deshacer (el deshacer es la clave).

Framework de 4 pasos

1
Clarificar
requisitos + restricciones

2
Diseñar
alto nivel (componentes + datos)

3
Profundizar
modelado + Flutter

4
Escalar
¿qué pasa con 1M usuarios?

Paso 1 — preguntas obligatorias: ¿cuántos usuarios? · ¿cuántos datos por día? · ¿latencia aceptable? · ¿requiere real-time (WebSocket vs polling)?

Paso 2 — componentes clave: Flutter app (cliente) → API (REST/WebSocket) → Backend (Supabase) → PostgreSQL → Cache (Redis / SharedPreferences).

Paso 4 — problemas comunes de escala

Problema	Solución
Demasiadas consultas	Caching (Redis, local)
Imágenes lentas	CDN + compression
Real-time	WebSocket / Supabase Realtime
BD lenta	Índices + read replicas
Servidor saturado	Load balancing

Ejemplo chat: tablas `conversations` · `messages` · `conversation_members` (PK compuesta) + suscripción con `onPostgresChanges(PostgresInsertEvent)` en `ChatCubit` → emite `Message.fromJson(payload.newRecord)`.

Errores en entrevistas

× codificar sin clarificar · × no mencionar escalabilidad (paso 4 siempre) · × ignorar edge cases · × solo REST cuando piden real-time (menciona WebSocket/Supabase Realtime).

8 Errores Comunes y Verificación

Fase	Error típico	Solución
Entender	No leer todo el enunciado · asumir sin preguntar	Leer 2 veces · preguntar edge cases · releer qué retorna (¿índices o valores?)
Approach	Brute force sin pensar · over-engineering · ignorar restricciones	Buscar patrón primero · solución más simple · ver límites antes de diseñar
Implementar	Off-by-one · índice inválido · olvidar edge cases · no deshacer en backtracking	Dibujar caso base · verificar bounds · testear vacío/1/2 · siempre <code>deshacer()</code>
Verificar	Solo mirar el código · solo happy path · "se ve bien"	Dry run con el ejemplo · probar vacío/1/máximo · trazar paso a paso

Regla de los 3 casos: 1) vacío/null → ¿no crashea? · 2) mínimo (1 elem) → ¿funciona? · 3) normal (ejemplo) → ¿retorna correcto?

```
// Errores clásicos en Dart
arr[0];           // X crash si vacío → arr.isEmpty ? null : arr[0]
map[key]!;        // X crash si falta → map[key] (null si no existe)
n * factorial(n - 1); // X sin caso base → infinito
backtrack(...);    // X olvidar deshacer → actual.removeLast()
```

Checklist antes de decir "estoy listo": ¿leí TODAS las restricciones? · ¿probé el ejemplo del enunciado? · ¿probé edge cases (vacío, 1 elem, null)? · ¿la complejidad cumple? · ¿el retorno es correcto? · ¿compila en Dart?

Los 10 ejercicios guiados — por patrón

#	Problema	Patrón	Complejidad
1	Two Sum (LC #1)	HashMap	$O(n) \cdot t \cdot O(n) \cdot e$
2	Valid Anagram (LC #242)	HashMap frecuencias	$O(n) \cdot t \cdot O(1) \cdot e$
3	Maximum Subarray (LC #53)	DP lineal (Kadane)	$O(n) \cdot t \cdot O(1) \cdot e$
4	Best Time to Buy/Sell (LC #121)	One-pass tracking mínimo	$O(n) \cdot t \cdot O(1) \cdot e$
5	Number of Islands (LC #200)	DFS/BFS en grid	$O(\text{rows} \times \text{cols})$
6	Coin Change (LC #322)	DP	$O(\text{amount} \times \text{coins})$
7	LRU Cache (LC #146)	HashMap + Doubly Linked List	$O(1)$ get/put
8	Merge K Sorted Lists (LC #23)	Heap (PriorityQueue)	$O(N \log k)$
9	Alien Dictionary (LC #269)	Topological Sort	$O(C)$ total chars
10	Word Ladder (LC #127)	BFS	$O(M^2 \times N)$

Los +25 adicionales por nivel (12)

Nivel	Problemas
Fácil (1-6)	Two Sum · Valid Anagram · Reverse String · Merge Sorted Arrays · Contains Duplicate · Max Subarray Sum
Medio (7-12)	Group Anagrams · Longest Consecutive Sequence · Product Except Self · Valid Parentheses · Binary Search Rotated · Climbing Stairs
Difícil (13-17)	Merge K Sorted Lists · LRU Cache · Word Ladder · Median of Two Sorted Arrays · Regex Matching
Dart específico (18-26)	Stack con List · Queue con List · HashMap simple · DFS in/pre/post-order · BFS por niveles · Graph BFS · Detectar ciclo (blanco/gris/negro) · Merge Sort · Quick Sort

Regla de los 20 minutos: 20 min sin avance → mira una pista (no la solución). 20 min más → revisa la solución y vuelve a intentarla mañana.

Progresión sugerida: Semana 1: 1-3 Easy (15-20 min c/u) · Semana 2: 4-5 Medium (30-45) · Semana 3: 6-7 Medium (45-60) · Semana 4: 8-10 Hard (60-90).

10 Recursos para Seguir Practicando

DESPUÉS DE ESTA GUÍA

Libros

Libro	Autor	Por qué
Competitive Programmer's Handbook	Laaksonen	Gratuito: fundamentos → DP, grafos, matemáticas
Principles of Algorithmic Problem Solving	Guzman	Metodología + ejemplos resueltos
Introduction to Algorithms (CLRS)	Cormen et al.	Referencia académica completa

Plataformas por nivel

Plataforma	Nivel ideal
Exercism (track Dart)	Principiante
Codewars	Principiante-Intermedio
HackerRank	Intermedio
LeetCode	Intermedio-Avanzado
Codeforces · CSES	Avanzado

Ruta entrevistas (3 meses): Mes 1: HackerRank Interview Prep Kit · Mes 2: LeetCode NeetCode 150 (patrón por patrón) · Mes 3: LeetCode Grind 75.

Ruta competitive (6 meses): CSES (Intro + Sorting + Graphs) → Codeforces Div 2 A-C → Div 2 D-E + AtCoder ABC.

Consejos de práctica: consistencia > cantidad (1 problema/día) · repite a los 30 días · explica en voz alta · respeta el timebox · agrupa por patrón, no por dificultad · trackea en una spreadsheet (problema, patrón, ¿lo resolviste solo?, días desde revisión).

A

Anexo · Índice del módulo (para profundizar)

Necesitas	Archivo
Framework de 6 pasos completo + ejemplo Valid Parentheses	01-metodologia-general.md
Big-O desde cero, regla del 10 ⁸ , 5 pasos con 4 ejemplos HackerRank	02-analisis-complejidad.md
Reconocimiento de patrones, keywords y árbol de decisión	03-reconocimiento-patrones.md
Referencia de estructuras de datos con complejidades	04-estructuras-datos-referencia.md
8 templates en Dart (Sliding Window → Backtracking)	05-patrones-avanzados.md
Comunicación en entrevistas y pseudocódigo limpio	06-comunicacion-y-pseudocodigo.md
10 ejercicios guiados (3 easy · 4 medium · 3 hard) con soluciones	07-ejercicios-practica.md
Recursión y backtracking (templates + memoización)	09-recursion-y-backtracking.md
System Design para Flutter (feed, chat, notificaciones)	10-system-design-basico.md
Errores comunes por fase + checklist de verificación	11-errores-comunes-patron.md
+25 ejercicios por nivel y dificultad	12-ejercicios-adicionales.md
Libros, plataformas, repos y canales de práctica	08-recursos-externos.md

Protocolo en una línea: Entender → Constraints → Ejemplos → Patrón → Pseudocódigo → Código. · Regla de oro: si puedes nombrar el patrón y su complejidad antes de codificar, ya tienes la mitad de la solución.